

Final Project Report

What did I do, and why is it interesting?

I trained keras encoder-decoder RNNs to predict movie titles based on movie summaries. For example, I can give these models the synopsis of *The Matrix* and they will guess its title. Their accuracy is low, but the strongest of these models gives grammatical titles that suit the movie about 20% of the time.

Even for a human, this task is only partially possible, because many movie titles aren't determined entirely by their plot summary, such as *Memento* or *Cloud Atlas*. The same movie *Cloud Atlas* could have been called *Every Time*, so no guesser can possibly guarantee the right answer.

What interests me about this task is seeing the model's guesses: informally, what does the model think, for example, *The Matrix* should be called? There are many reasonable answers: "The Matrix," "Neo," "The One," "Escape the Matrix," and so on. As it happens, my best model does give a reasonable response here: "The One."

What is my data?

The training and development data is from [this Kaggle dataset](#), which contains data for about 50,000 movies: the title, genre, and IMDb's summary. All of this was scraped from IMDb's public-facing pages, where IMDb users can input and vote on candidate summaries. Titles are 2.3 tokens on average, and summaries are 101 tokens on average. The summaries contain 170,000 unique tokens, and the titles contain 46,000 unique tokens. Both are open classes. I used this entire Kaggle dataset for this project: the first 80% as training data, and the remaining 20% as development data.

My test data is a set of 20 movies I know, but which are not in the Kaggle set: their titles, and IMDb's summaries of them, which I copied from IMDb's public pages. These movies are hand-picked, which is a bias in this evaluation, but in my view, this task is inherently subjective, so I must use movies I happen to know.

How do my models work?

All the data is featurized into sequences of embeddings. The embeddings are GloVe pre-trained word embeddings. One of my models uses 50-dimensional embeddings and the other uses 300-dimensional embeddings. But importantly, I augment these embeddings by three dimensions (to 53 and 303) – the extra dimensions are used to make unique START, STOP, and FILLER embeddings, which are unit vectors in the respective extra dimensions. All embeddings of other words are padded with three zeros.

Each title is converted into a sequence of word-embeddings, which is then bookended with a START and STOP embedding. Keras training wants numpy arrays, so all training data is padded with FILLER tokens to a constant length. For titles, this length is 10. As an example, the featurized title for *The Matrix* would become (representing word embeddings as words in brackets):

[START, [the], [matrix], STOP, FILLER, FILLER, FILLER, FILLER, FILLER, FILLER]

Summaries are featurized in a variety of ways. Optionally, stop words are removed. Then, the summary is converted into a sequence of either word-embeddings or sentence-embeddings. For word-embeddings, the constant length is 100, and for sentence embeddings, the constant length is 10. As an example, if word embeddings are used, the featurized summary of The Matrix would look like this:

[[thomas], [anderson], [is], [a], ... FILLER, FILLER] (*100 elements*)

If sentence embeddings are used, it would look like this:

[[thomas anderson is a...], [by day he is...], [neo has always...], ... FILLER] (*10 elements*)

Sentence embeddings are calculated as the average of the embeddings of its words.

All my models are encoder-decoder RNNs with an LSTM. The encoder and decoder are both just one input layer, one recurrent layer with an LSTM, and one output layer. The following two paragraphs about the models' inner workings are based on the course textbook chapter 9, and a Keras sequence-to-sequence tutorial, as well as my own experience building this project.

During training, the encoder is given the summary embedding-by-embedding, and its output is ignored. Then the decoder is given the title embedding-by-embedding, starting with the START embedding, and it's trained to predict each next embedding of the title.

During inference, the encoder is given the summary as before, and then the decoder is given a START embedding. Each embedding it outputs is interpreted as the nearest token in the vocabulary (by cosine similarity), and then this token's actual embedding is used as the next input embedding. This is repeated until the decoder predicts a STOP or FILLER token. The predicted title is the sequence of nearest tokens.

Unknown words are always skipped. All embeddings given to the model are normalized. The activation function in the decoder's output layer uses tanh, so it can output any value in (-1, +1).

I experimented with many configurations of the four hyperparameters (word embeddings vs. sentence embeddings, keep stops vs. remove them, embedding size, and hidden layer size). Ultimately, I settled on two models:

"Simple" is the model with simplest configuration possible: word embeddings, keep stop words, 50-dimension embeddings, 32 hidden units. It has 23,675 parameters, and training took about 30 minutes.

"Fancy" is the strongest model I could make, which turned out to be: sentence embeddings, keep stop words, 300-dimension embeddings, 256 hidden units. Removing stop words slightly worsens performance, and additional hidden units beyond 256 only slow down training and inference without improving performance. It has 1,224,751 parameters, and training took about 10 minutes.

What are my results/conclusions?

I evaluated these models in two ways: statistically using the dev set, and subjectively using a small, hand-picked test set of movies.

Statistical evaluation: Accuracy

I expected a low accuracy, perhaps as high as 1%. In reality, both models, tested over the entire dev set, have an accuracy of 0%. Neither one ever predicted the exact real title.

Statistical evaluation: Cosine similarity

First, I'll describe my baseline, which is a "model" that always predicts the exact average of the entire test set. This average turns out close to:

[[the], STOP, FILLER, FILLER, FILLER, FILLER, FILLER, FILLER, FILLER, FILLER]

Across the dev set, the average cosine similarity of this prediction is 0.68 for 50-dimension embeddings, and 0.65 for 300-dimension embeddings¹.

"Simple," which uses 50-dimensional embeddings, achieves an average cosine similarity of 0.7988, which is 17% higher than the baseline of 0.68.

"Fancy," which uses 300-dimensional embeddings, achieves an average cosine similarity of 0.7613, which is also 17% higher than its baseline of 0.65.

As a point of comparison, the cosine similarity of "gray" and "blue" in these GloVe word embedding models is 0.78 for 50-dimensional embeddings, and 0.55 for 300-dimensional embeddings. Using this as a comparison, "Simple" typically predicts a next token that is slightly more similar to the target token than "gray" is to "blue," and "Fancy" typically predicts a next token that is substantially more similar to the target token than "gray" is to "blue."

However, keep in mind that most target tokens are STOP and FILLER. Most of the loss of these models comes from its predictions of the actual words of the title, and it's often very accurate when predicting FILLER tokens.

Subjective evaluation

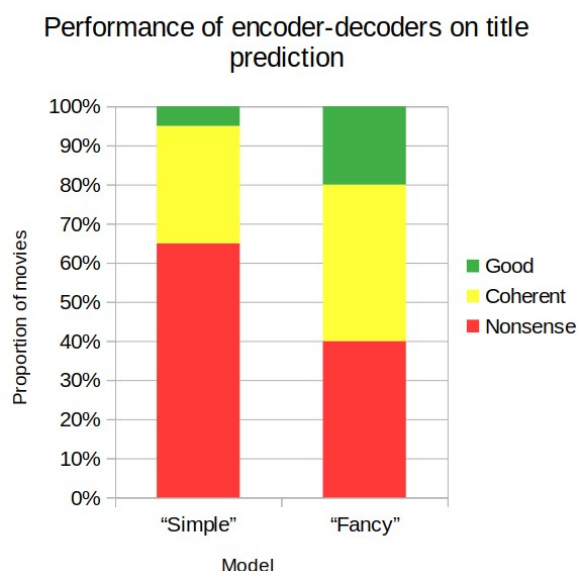
As I said above, the fun of this project is seeing what the model predicts. I hand-picked a set of 20 movies I know, which aren't in the training or dev sets. Then I polled each model for its predicted title for each summary. The predictions are here:

1 More dimensions leads to a worse baseline here, because in N dimensions there are N places for error to accumulate.

Real title	Simple's prediction	Fancy's prediction
Snow White and the Seven Dwarfs	Little Little Little	The Love of the Beautiful
The Godfather	One One One	The Man
The Matrix	One One One	The One
Cloud Atlas	The One One	The One
Star Wars: Episode IV – A New Hope	The Beast One	The Man One
The Wizard of Oz	The Ghost One	The Man One
Memento	One Life. One	The One
The Lord of the Rings: The Fellowship of the Ring	The Ghost the Ghost	The Ghost of the Ghost
Portrait of a Lady on Fire	La Dans La La	La La La La
Citizen Kane	One One One	The One
Inception	One One	The One
Arrival	The One One	The One
Coco	La Voz	La De La Noche
Spirited Away	Love One Life	The So So So
Shrek	Dragon Cat	The Man But So So
Gone with the Wind	One Life. One	The One One
Finding Nemo	The Ghost the Ghost	The One One
WALL-E	Into the Into	The Just Just Just
Psycho	One One One	The Just Just
The Danish Girl	"One One One	Der Und Und Und Und Und

I've colored the predicted titles by their subjective accuracy: Green means a good title for that movie, yellow means a coherent title for some movie, and red means an incoherent title. This evaluation is subjective: someone else might say that "The Man One" isn't a coherent title, or that "One One One" is a coherent title, or that "The Man" isn't a fitting title for *The Godfather*, etc.

This graph summarizes these two models' performance on this small test set:



To me, subjectively, the highlights of this small test set are:

- Both models are sensitive to language cues in the summary:
 - For *Coco*, which is set in Mexico on Día de Muertos, both models give Spanish titles.
 - For *Portrait of a Lady on Fire*, which is set in France, both models give French titles.
 - For *The Danish Girl*, which is set in Denmark, “Fancy” gives a German title.
- “Fancy” has learned some basic features of typical titles:
 - Many titles start with “The” (or “La,” or “Der”).
 - Many titles fit the pattern “The X of the Y.”
 - Many titles use simple, common nouns like “One,” “Man,” and “Love.”
- “Fancy” gets a lot of credit here for simply predicting “The Man” and “The One” often, because many movies are about a man or a unique person.

Future experiments/explorations/additions?

The biggest single improvement I can see for this project is using more data. Only 50,000 movies probably just isn’t enough data for any encoder-decoder to learn to interpret English. IMDb has records for over a million movies, according to the Wikipedia page for IMDb, which may be enough.

A related possible improvement is to use a pre-trained language model such as GPT-2. Professor Felix suggested I consider using a pre-trained model, but I thought it would be more interesting to build from scratch (or rather, only using pre-trained embeddings). I still think it was more interesting, but a model pre-trained with English should be able to perform much better on this task.

Works Cited

- The Wikipedia page for IMDb: <https://en.wikipedia.org/wiki/IMDb>
- My Kaggle dataset: <https://www.kaggle.com/datasets/hijest/genre-classification-dataset-imdb/>
- The course textbook, *Speech and Language Processing*: <https://web.stanford.edu/~jurafsky/slp3/>
- Two Keras sequence-to-sequence tutorials I relied closely on, for code and understanding:
 - <https://blog.keras.io/a-ten-minute-introduction-to-sequence-to-sequence-learning-in-keras.html>
 - https://keras.io/examples/nlp/lstm_seq2seq/